CS 301 · MEETING 14 · ACT 0

Movie Ratings as a Matrix

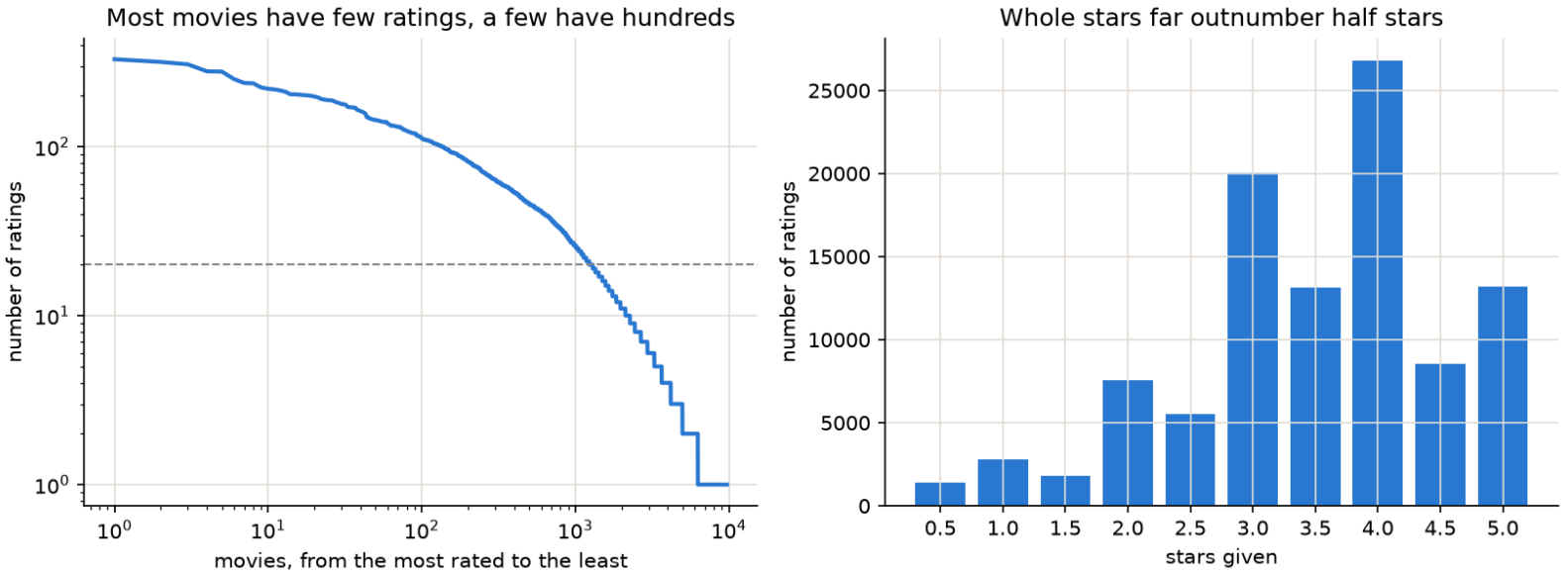
The rating matrix, missing entries and movie features computed with matrix products.

Ratings as matrix entries

```
ratings: 100,836 rows and 4 columns
   userId  movieId  rating  timestamp
0       1        1     4.0  964982703
1       1        3     4.0  964981247
2       1        6     4.0  964982224
```

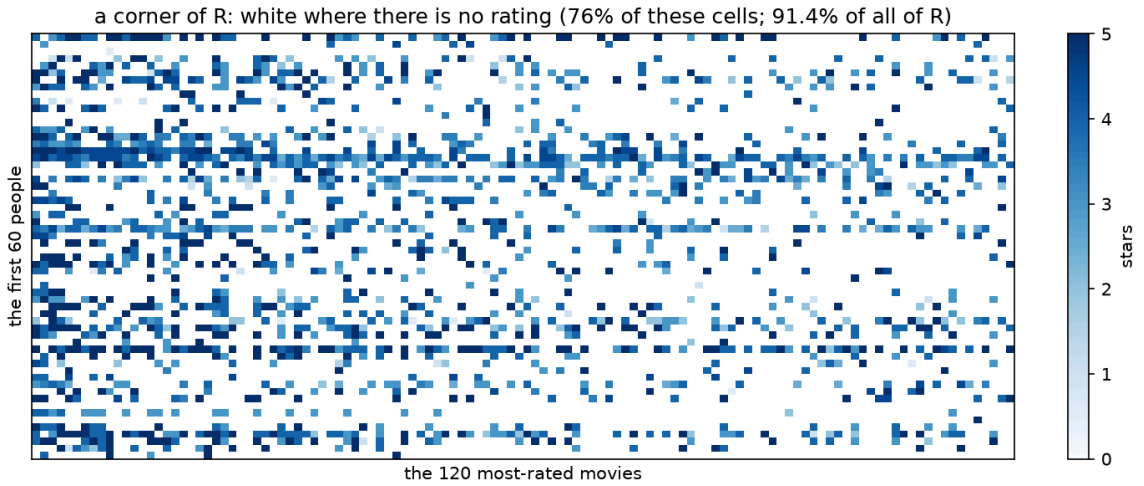
- MovieLens: 610 people, 9,724 movies, 100,836 ratings, half a star to five
- Each rating supplies a person ID, a movie ID and a value
- Map the IDs to row and column indices for **COO format**: 100,836 entries in 5.9 million cells

The long tail



- Half of the movies have three ratings or fewer; a few have hundreds
- Keep the movies with **20 or more**: 1,297 of them, two thirds of all the ratings

R, a matrix with holes




```
R has 610 rows (people) and 1,297 columns (movies): 791,170 cells, 67,898 of them with a rating
```

- `R`: 610 rows, 1,297 columns, `NaN` where there is no rating
- `B = ~np.isnan(R)`: True where there is one; 8.6% of the cells
- **A missing rating is not zero stars**

Movie features from column sums

$$\text{count}_j = \sum_i B_{ij} = (1^T B)_j, \qquad \text{mean}_j = \frac{(1^T R_0)_j}{\text{count}_j}$$

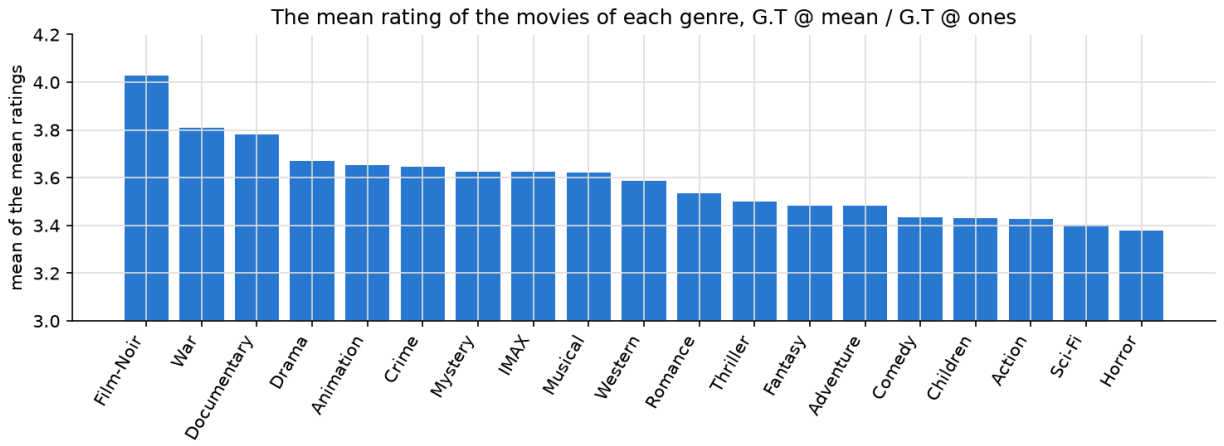
```
ones = np.ones(len(people))
count = ones @ B                          # number of observed ratings per movie
R0 = np.nan_to_num(R)                     # zero-fill for summing observed ratings
mean = (ones @ R0) / count
```

- A **feature** of a movie: one number computed from its column
- A column sum is a product with a vector of ones
- Divide each sum by its observed rating count

Mean rating by genre

$$\text{mean per genre} = \frac{G^T \,\text{mean}}{G^T\, 1}$$

```
G: 1,297 movies by 19 genres, 1 where the movie has the genre
mean_per_genre = (G.T @ mean) / (G.T @ ones_movies)
```



- `G.T @ mean` sums movie means; `G.T @ ones_movies` counts movies in each genre
- Each movie counts once in each of its genres